SystemVerilog Assertions (SVA)

Concepts, Examples, and Interview Questions

Kittu K Patel
ASIC Verification Engineer

July 26, 2025

Contents

1 Introduction to SystemVerilog Assertions

SystemVerilog Assertions (SVA) are used to validate the dynamic behavior of digital designs. They help catch functional bugs early during simulation by monitoring design behavior against specified properties.

Assertions can be:

- **Immediate**: Checked immediately within procedural blocks.
- **Concurrent**: Evaluated over multiple clock cycles to monitor temporal behaviors.

SystemVerilog assertions are a powerful verification aid for both simulation and formal verification environments. The syntax and semantics are governed by the IEEE 1800 SystemVerilog LRM (Language Reference Manual).

2 Syntax Overview

2.1 Immediate Assertion

Executed as soon as the statement is reached during simulation.

Listing 1: Example: Immediate Assertion

```
assert (a == b) else $error("Mismatch between a and b");
```

2.2 Concurrent Assertion

Used to monitor behavior across multiple cycles and are always associated with a clocking event.

Listing 2: Syntax: Concurrent Assertion with property

```
property prop_name;
    @(posedge clk) req |=> ##[1:3] gnt;
endproperty

assert property (prop_name);
```

3 Basic Operators in SVA

- `|=>` : Implication (non-overlapping)
- `|->` : Implication (overlapping)
- `n` : Delay operator
- `[*n]` : Repetition
- `within` : Occurrence within time window
- `disable iff` : Condition to ignore assertion

4 Detailed Example: Request-Grant Protocol

4.1 Description

Ensure that when a request signal is asserted, a grant is issued within 3 clock cycles.

4.2 Property and Assertion

Listing 3: Request-Grant Property and Assertion

```
property p_request_grant;
    @(posedge clk)
    req |=> ##[1:3] gnt;
endproperty

assert property (p_request_grant)
    else $error("Grant not received within 3 cycles of request");
```

4.3 Explanation

- `|=>` is used for non-overlapping implication.
- `[1:3]` allows grant to occur after 1 to 3 clock cycles.
- If the grant does not arrive within that range, the assertion fails and the error is logged.

5 Concurrent Assertions

5.1 Introduction

Concurrent assertions, also known as temporal assertions, are used in SystemVerilog to specify behaviors that span over time. These assertions are evaluated in parallel with simulation time and are a powerful way to express timing relationships between events.

SystemVerilog provides the following constructs for concurrent assertions:

- `assert property`
- `assume property`
- `cover property`

Concurrent assertions are typically written using temporal logic operators such as `##`, `[*]`, `->`, `until`, `disable iff`, and more.

5.2 Property Specification

A property is a reusable expression of a temporal behavior. You define it using the `property` keyword and later bind it to an assertion using `assert property`.

```
property prop_name;
  @(posedge clk) req |-> ##[1:3] ack;
endproperty

assert property (prop_name);
```

5.3 Temporal Operators (from LRM)

- `->`: Implication. If the LHS is true, then the RHS must be true in the next cycle.
- `|->`: Overlapped implication. LHS and RHS can be true in the same cycle.
- `n`: Delayed sequence. Specifies a delay of `n` clock cycles between events.
- `[*n:m]`: Repetition. A sequence should repeat `n` to `m` times.
- `disable iff (cond)`: Temporarily disables assertion evaluation if the condition is true.

5.4 Example 1: Handshake Protocol

```
property handshake;
  @(posedge clk)
  req |-> ##[1:2] ack;
endproperty

assert property (handshake);
```

Explanation: If `req` is asserted, then `ack` must be asserted 1 or 2 cycles later.

5.5 Example 2: Reset-Safe Assertion

```
property safe_access;
  @(posedge clk)
  disable iff (reset)
  enable |-> ##1 ready;
endproperty

assert property (safe_access);
```

Explanation: This assertion is disabled during reset. When `enable` is asserted, `ready` must be asserted in the next cycle.

5.6 Example 3: FIFO Write-Read Constraint

```
property fifo_read_after_write;
  @(posedge clk)
  write |-> ##[2:4] read;
endproperty

assert property (fifo_read_after_write);
```

Explanation: After a `write`, a `read` must occur between 2 to 4 cycles.

5.7 Use of `cover property`

```
cover property (@(posedge clk) req |-> ##1 ack);
```

Explanation: This line records coverage whenever the specified property holds true. It's useful in testbench functional coverage.

5.8 When to Use Concurrent Assertions

- To monitor timing relationships across multiple clock cycles.
- To verify protocol rules in a design.
- To ensure correct sequence of events in control logic or FSMs.
- To capture coverage on events that span time.

5.9 Tips for Writing Concurrent Assertions

- Use named properties for reusability and clarity.
- Always use `disable iff` for assertions with reset.
- Use bounded delays (`[n:m]`) instead of fixed ones when timing may vary.
- Avoid complex expressions inside assertions; break them down for better readability.

6 Immediate Assertions

6.1 Introduction

Immediate assertions in SystemVerilog are used to check conditions that must hold at a specific simulation time. They are evaluated at the time they are encountered during simulation, typically inside procedural blocks like `initial`, `always`, or `always_ff`.

Immediate assertions are useful for validating single-time-point conditions such as setup, initialization, and functional constraints within a single cycle.

6.2 Types of Immediate Assertions

There are two types of immediate assertions:

- `assert (expression);`
- `assume (expression);`

`assert` is used for verification, and `assume` is used mainly in formal verification for guiding tools.

6.3 Syntax

```
assert (expression)
  else $fatal("Assertion failed");
```

Note: The `else` clause is optional. Without it, a failure will print a default error.

6.4 Example 1: Checking Valid Range

```
always @(posedge clk) begin
  assert (addr < 1024)
    else $fatal("Address out of range at time %0t", $time);
end
```

Explanation: Ensures that the `addr` signal never exceeds the limit of 1023 at any positive clock edge.

6.5 Example 2: Initialization Check

```
initial begin
  assert (reset == 1'b1)
    else $fatal("Reset not asserted at time zero");
end
```

Explanation: Checks that the reset is active at simulation start.

6.6 Use Cases of Immediate Assertions

- Checking combinational logic conditions.
- Validating parameter constraints at compile or elaboration time.
- Ensuring FSMs or control logic start in a defined state.
- Debugging unexpected behavior with clear failure messages.

6.7 Advantages

- Simple and intuitive syntax.
- Useful for single-time condition validation.
- Allows integration of `$display`, `$fatal`, `$error`, or even `$stop` for debugging.

6.8 Best Practices

- Use meaningful failure messages with `$fatal`, `$error`, or `$display`.
- Avoid side-effects within assert expressions.
- Use assertions even in testbenches to catch coverage misses or incorrect stimuli.
- Group assertions logically and document them thoroughly.

6.9 Comparison with Concurrent Assertions

Immediate Assertions	Concurrent Assertions
Evaluated at the time of execution	Evaluated over a span of time
Used inside procedural blocks	Used outside procedural blocks
Checks instantaneous conditions	Checks temporal behaviors
Simple and fast to simulate	Slightly heavier but more powerful

7 Concurrent Assertions

7.1 Introduction

Concurrent assertions in SystemVerilog are used to monitor and validate sequences of events over time. Unlike immediate assertions which evaluate instantly at simulation time, concurrent assertions operate over multiple simulation cycles, making them ideal for checking protocol behaviors and temporal relationships between signals.

They are typically used in testbenches or verification IP to model and check expected system behavior.

7.2 Syntax Overview

Concurrent assertions are declared using the `assert property`, `assume property`, or `cover property` constructs.

```
property prop_name;
  @(posedge clk) disable iff (!reset) <sequence_expression>;
endproperty

assert property (prop_name)
  else $error("Assertion Failed: prop_name");
```

7.3 Key Constructs

- **property** - Defines a temporal relationship to be checked.
- **sequence** - Used within properties to describe signal activity patterns.
- **assert property** - Checks the property during simulation.
- **cover property** - Measures coverage of specific properties.
- **assume property** - Assumptions for formal verification.

7.4 Example 1: Simple Handshake Protocol

```
property handshake;
```

```
  @(posedge clk)
  disable iff (!reset)
  req |=> ack;
endproperty

assert property (handshake)
  else $fatal("ACK not asserted after REQ");
```

Explanation: This assertion checks that if `req` goes high, then `ack` must follow in the next clock cycle.

7.5 Example 2: Delayed Signal Response

```
property write_delay;
  @(posedge clk)
  disable iff (!reset)
  wr_en ##1 data_valid;
endproperty

assert property (write_delay)
  else $fatal("Data valid not asserted one cycle after write enable");
```

Explanation: Ensures `data_valid` is high exactly one cycle after `wr_en`.

7.6 Sequence Operators in Concurrent Assertions

- `|=>` : Implication (one cycle after)
- `n` : Delayed cycles (n-clock delay)
- `[*]` : Zero or more repetitions
- `[+]`, `[=n]`, `[->n]` : Event repetitions
- `and`, `or`, `intersect` : Sequence composition

7.7 disable iff Clause

The `disable iff` clause temporarily disables the assertion when the reset condition is active. It prevents false assertion failures during system reset or initialization.

```
disable iff (!reset)
```

7.8 Example 3: Read Followed by Acknowledge Within 3 Cycles

```
property read_acknowledge;
  @(posedge clk)
  disable iff (!reset)
  read_req |=> ##[1:3] read_ack;
endproperty

assert property (read_acknowledge)
```

```
else $fatal("ACK not received within 3 cycles of read request");
```

7.9 Best Practices

- Always use `disable iff` to guard against unwanted behavior during reset.
- Use meaningful assertion names and error messages.
- Separate assertions from DUT to improve modularity and reuse.
- Combine assertions with functional coverage for completeness.

7.10 Concurrent vs Immediate Assertions

Concurrent Assertions	**Immediate Assertions**
Evaluated over time	Evaluated at current time
Can monitor protocol behavior	Checks one-time conditions
Supports temporal logic (, —=¿)	No support for time delay
Used outside procedural blocks	Used inside procedural blocks

SystemVerilog Assertion Interview Questions (With Hints)

1. [**Intel**] What is the difference between an `immediate` and a `concurrent` assertion in SystemVerilog?
   Hint: Think about when and how each gets triggered.

2. [**Nvidia**] Write an assertion to check that once `req` goes high, `gnt` should go high within 3 cycles.
   Hint: Use the `|=>` implication with a range.

3. [**Micron**] What is the difference between and `[*]` in SystemVerilog assertions?
   Hint: One deals with clock cycles, the other with repetition.

4. [**Synopsys**] Explain vacuous success and how it affects assertion coverage.
   Hint: Consider what happens when antecedents are never true.

5. [**Qualcomm**] Write an assertion to ensure signal `data_valid` stays high for exactly 4 cycles after `start` is asserted.
   Hint: Use `start |=> data_valid[*4]`.

6. [**Broadcom**] How do `disable iff` and `not` differ in filtering assertion behavior?
   Hint: One affects execution, the other logic.

7. [**AMD**] What happens when an assertion is triggered in a simulation? What options are available to control the response?
   Hint: Think about `severity levels` and simulation control.

8. [**Texas Instruments**] Explain the role of *rose*(),fell(), and *stable*()*inassertions*.
   Hint: These are sampling expressions.

9. [**ARM**] Write a SystemVerilog assertion to check that signal `ack` stays low during reset.
   Hint: Use `always` *block with* `disable iff (reset)`.

10. [**Apple**] How would you monitor a protocol where one signal must not change until another toggles?
    Hint: Use `throughout` *or* `until` *operators.*